Distributed Document Search Service

Architecture Design Document

1. Objective and Assumptions

This prototype is designed to demonstrate how I would build a **multi-tenant, horizontally scalable, low-latency document search service** that can evolve toward handling **10M+ documents**, **1000+ searches/sec**, and **sub-500ms p95** search latency.

Functional goals

- Index and search documents across multiple tenants
- Support full-text search with relevance ranking
- Retrieve documents by ID
- Delete documents and remove them from the search index
- Enforce tenant isolation
- Provide basic rate limiting and health checks

Non-functional goals

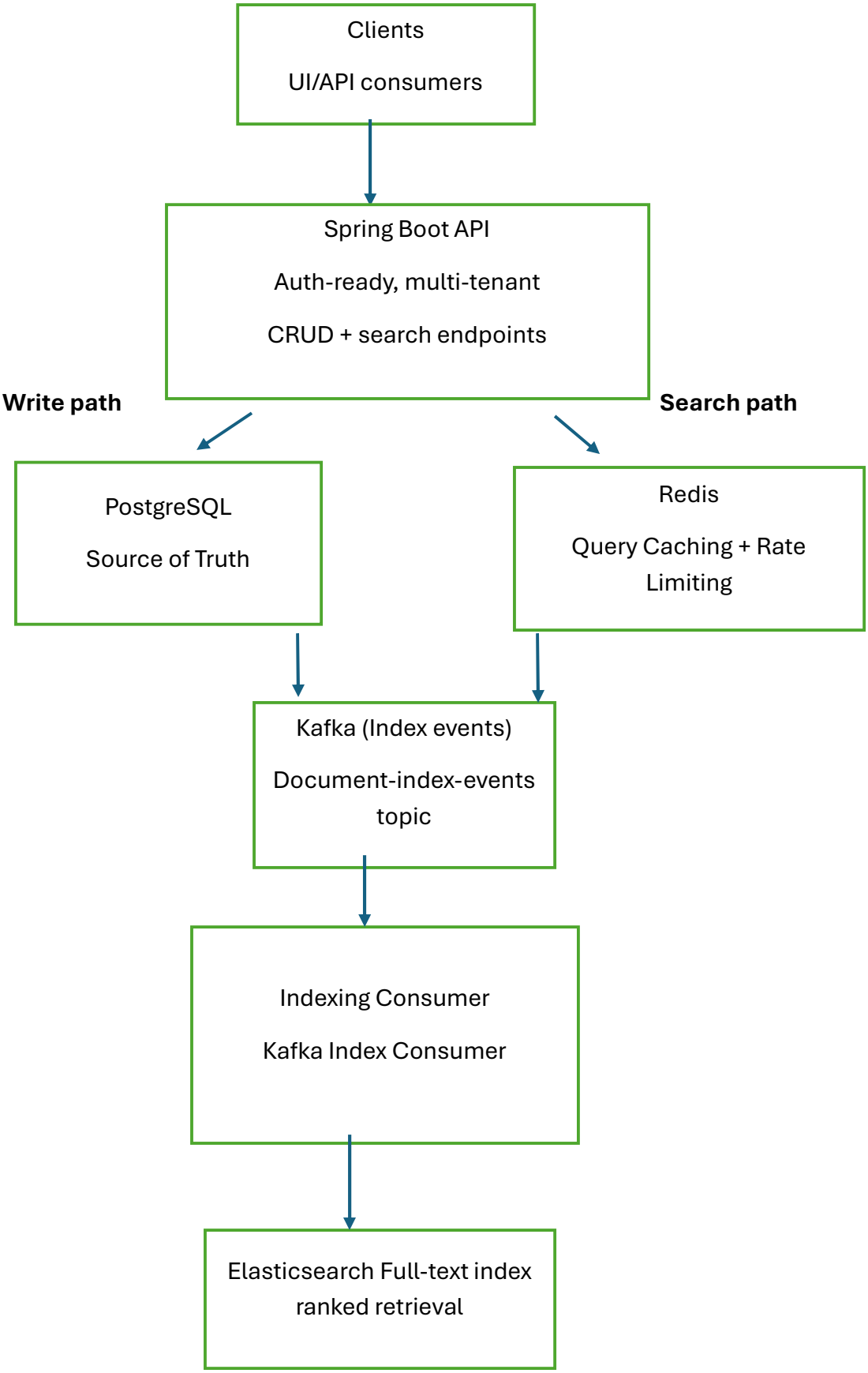
- Sub-500ms p95 query latency for hot/common queries
- Horizontal scalability at API, consumer, cache, and search tiers
- Fault isolation between write path and search path
- Clear production evolution path

Assumptions

- Document sizes are moderate text payloads
- Search is eventually consistent relative to writes
- Tenant ID is provided via X-Tenant-ID header
- Prototype prioritizes architectural clarity over complete production hardening

---

2. High-Level Architecture



Design intent

The system separates:

- **system of record** concerns → PostgreSQL
- **search/read optimization** concerns → Elasticsearch
- **asynchronous integration** concerns → Kafka
- **hot query + request control** concerns → Redis

This decoupling keeps the system maintainable and scalable.

---

3. Data Flow

3.1 Indexing / Write Flow

```
POST /documents
  |
  v
API validates tenant + request
  |
  v
Persist document in PostgreSQL
  |
  v
Publish UPSERT event to Kafka
  |
  v
Kafka consumer receives event
  |
  v
Read document from PostgreSQL
  |
  v
Index into Elasticsearch
  |
  v
Future searches can discover document
```

Why this flow

PostgreSQL is the **source of truth**, while Elasticsearch is an optimized read model. Using Kafka decouples indexing from synchronous API response time and prevents Elasticsearch latency from directly impacting writes.

Consistency implication

A newly created document may be visible in /documents/{id} before it becomes searchable in /search. This is an intentional **eventual consistency** trade-off to improve write latency and system resilience.

3.2 Search Flow

```
GET /search?q=... + X-Tenant-ID
  |
  v
Apply rate limit per tenant
  |
  v
Check Redis cache for query + tenant key
  |
  +---- cache hit ----> return cached response
  |
  +---- cache miss ---> query Elasticsearch
                                  |
                                  v
            apply tenant filter + full-text search
                                  |
                                  v
            rank results and return response
                                  |
                                  v
            cache non-empty result in Redis
```

Why this flow

Most repeated enterprise search traffic is skewed toward:

- repeated popular terms
- dashboard-driven requests
- tenant-specific recurring searches

Redis absorbs these repeated hot reads and protects Elasticsearch from unnecessary load.

---

4. Storage and Database Strategy

4.1 PostgreSQL

Role: transactional source of truth

Why PostgreSQL

- Strong transactional guarantees
- Easy CRUD semantics
- Mature operational tooling
- Reliable storage for canonical documents
- Good fit for metadata, tenant ownership, timestamps, and future relational joins

What it stores

- document ID
- tenant ID
- title
- content
- created/updated timestamps

Why not use Elasticsearch as primary storage

Elasticsearch is excellent for search, but it is not ideal as the system of record for transactional writes, tenant ownership validation, and long-term canonical persistence.

4.2 Elasticsearch

Role: full-text search engine and ranking layer

Why Elasticsearch

- BM25 relevance ranking out of the box
- Inverted index optimized for large-scale text search
- Low-latency retrieval
- Easy support for future fuzzy search, highlighting, facets, autocomplete
- Horizontal shard-based scaling

Search design

Each indexed document contains:

- id
- tenantId
- title
- content

Tenant filtering is applied during search so that a tenant can only retrieve its own documents.

Query strategy

Prototype query uses:

- multi_match across title and content
- title boosted higher than body content
- tenantId exact filter

This gives reasonably good relevance while remaining simple.

4.3 Redis

Role: cache + rate limiter state

Why Redis

- Extremely low-latency key/value access
- TTL-based cache semantics
- Easy fit for per-tenant counters
- Common choice for hot query caching and throttling

What it stores

- search response cache entries
- rate-limit counters per tenant

Example keys

- search:{tenant}:{queryHash}:{size}
- rl:search:{tenant}:{window}

4.4 Kafka

Role: asynchronous messaging for indexing

Why Kafka

- Decouples write API from indexing work
- Smooths bursts of write traffic
- Enables retry and replay patterns
- Natural evolution path to multiple consumers / partitions
- Strong fit for event-driven read model projection

Topic usage

Prototype uses one topic:

- document-index-events

Event types:

- UPSERT
- DELETE

---

5. API Design

5.1 Create Document

Endpoint

POST /documents
X-Tenant-ID: tenant-a
Content-Type: application/json

Request

```
{
  "title": "Distributed systems notes",
  "content": "Kafka Elasticsearch Redis scaling"
}
```

Response

```
{
  "id": "15eb4261-5d29-4c6d-83cc-a7ceaf291e9b",
  "tenantId": "tenant-a",
  "title": "Distributed systems notes",
  "content": "Kafka Elasticsearch Redis scaling",
  "createdAt": "2026-04-21T15:17:20.133021930Z",
  "updatedAt": "2026-04-21T15:17:20.133021930Z"
}
```

5.2 Search

Endpoint

GET /search?q=distributed&size=10
X-Tenant-ID: tenant-a

Response


```
{
  "tenantId": "tenant-a",
  "query": "distributed",
  "total": 1,
  "results": [
    {
      "id": "15eb4261-5d29-4c6d-83cc-a7ceaf291e9b",
      "tenantId": "tenant-a",
      "title": "Distributed systems notes",
      "snippet": "Kafka Elasticsearch Redis scaling",
      "score": 0.5753642
    }
  ],
  "cacheHit": false
}
```

5.3 Get Document

Endpoint

GET /documents/{id}
X-Tenant-ID: tenant-a

Response


```
{
  "id": "15eb4261-5d29-4c6d-83cc-a7ceaf291e9b",
  "tenantId": "tenant-a",
  "title": "Distributed systems notes",
  "content": "Kafka Elasticsearch Redis scaling",
  "createdAt": "2026-04-21T15:17:20.133021Z",
```

```
  "updatedAt": "2026-04-21T15:17:20.133021Z"
}
```

5.4 Delete Document

Endpoint

```
DELETE /documents/{id}
X-Tenant-ID: tenant-a
```

Response

204 No Content

5.5 Health Check

Endpoint

```
GET /health
```

Response


```
{
  "status": "UP",
  "dependencies": {
    "kafka": "UP",
    "redis": "UP",
    "postgres": "UP"
  }
}
```


---

6. Consistency Model and Trade-Offs

Chosen model

- **Strong consistency** for document storage in PostgreSQL
- **Eventual consistency** for search visibility in Elasticsearch

Why this is the right trade-off

Search systems at scale typically separate:

- durable transaction processing
- search-optimized indexing

If I required fully synchronous dual writes to Postgres and Elasticsearch before acknowledging the request:

- API latency would increase
- write availability would depend on search cluster health
- transient ES failures would directly break writes

Instead, Kafka decouples those concerns.

Trade-off

A document may be:

- immediately readable by ID
- not instantly searchable for a short time window

This is acceptable for most enterprise search systems and is explicitly documented.

---

7. Caching Strategy

7.1 Search Result Caching

Search responses are cached in Redis using a tenant-aware key. This prevents cross-tenant leakage and avoids repeated ES queries for common searches.

Policy

- cache only non-empty results
- TTL = 60 seconds in prototype
- evict tenant search keys on document create/delete

Why cache only non-empty results

Caching empty results in an eventually consistent system can create stale misses right after indexing. Avoiding empty-result caching improves correctness.

7.2 Future Cache Layers

In production I would add:

- application-level short in-memory cache for very hot local results
- CDN/API gateway caching for safe public metadata if applicable
- Elasticsearch query result tuning and request cache where appropriate

---

8. Message Queue Usage

Kafka is used to offload indexing work from the synchronous write path.

Benefits

- isolates write latency from indexing latency
- absorbs spikes in document ingestion
- supports replay/recovery
- enables independent scaling of API and indexer services

Current prototype choice

For deterministic demo behavior, the prototype uses:

- single topic
- single partition
- single consumer concurrency

Production evolution

At scale I would move to:

- multiple partitions
- multiple consumer replicas
- dead-letter topic
- replay tooling
- lag monitoring and autoscaling

---

9. Multi-Tenancy and Isolation Strategy

Multi-tenancy is enforced at **multiple layers**, not just the API boundary.

9.1 Tenant identification

Tenant is passed in:

X-Tenant-ID

9.2 Isolation points

Isolation is enforced in:

- API request handling
- PostgreSQL retrieval by tenant
- Elasticsearch filter on tenantId
- Redis cache key namespace
- Redis rate-limit key namespace

Why this matters

Relying on only one layer is dangerous. A secure enterprise design should defend against:

- application bugs
- accidental cache pollution
- search query mistakes
- cross-tenant result leakage

This prototype demonstrates layered isolation clearly.

---

10. Fault Tolerance and Horizontal Scalability

10.1 API Layer

Spring Boot service is stateless and can scale horizontally behind a load balancer.

10.2 Kafka Consumer Layer

Indexing consumers can scale independently from API instances. In production, partition count would be increased to parallelize indexing throughput.

10.3 Elasticsearch

Elasticsearch can scale by:

- sharding indices
- adding data nodes
- tuning refresh interval
- using replica shards for higher read throughput and availability

10.4 Redis

Redis can evolve from a single node to:

- replication
- sentinel
- cluster mode

10.5 PostgreSQL

For 100x scale, likely evolution:

- read replicas
- partitioning by tenant or time
- connection pooling
- eventually, metadata/data separation if needed

---

11. Design Decisions and Rationale

Why Postgres + Kafka + Elasticsearch + Redis

This architecture is intentionally common in enterprise systems because each component is specialized:

- Postgres → correctness

- Kafka → decoupling
- Elasticsearch → search performance
- Redis → low-latency repeated access

This makes trade-offs explicit and provides a realistic production path.

Why not use only PostgreSQL full-text search

PostgreSQL FTS can work for smaller systems or lower-scale prototypes, but for the stated goals:

- 10M+ docs
- sub-second search
- high concurrency
- richer relevance and future search features

Elasticsearch is the better long-term fit.

---

13. Production Evolution Summary

To make this production-ready, I would next implement:

- authentication and authorization using JWT/OAuth2
- tenant-to-user authorization checks
- outbox pattern or CDC for transaction-safe event publication
- retry policies and dead-letter queues
- Prometheus + Grafana + distributed tracing
- Elasticsearch shard and lifecycle management
- blue-green or rolling deployments
- benchmark-driven tuning
- backup/recovery automation
- SLO/SLA dashboards and alerting

---

14. Conclusion

This design intentionally separates transactional storage from search optimization and uses an event-driven indexing pipeline to achieve scalability, resilience, and maintainability. The prototype demonstrates the essential enterprise patterns needed for a durable search platform:

- multi-tenant isolation
- asynchronous indexing
- low-latency search
- cache-assisted performance
- horizontally scalable service boundaries
- explicit consistency trade-offs

15. Production Readiness Analysis

This prototype demonstrates the right architectural shape for a distributed search platform, but several areas would need to be strengthened before production use. The goal in production would be to preserve the same separation of concerns—PostgreSQL for canonical data, Kafka for asynchronous propagation, Elasticsearch for search, Redis for caching and throttling—while improving scalability, fault tolerance, security, observability, and operational maturity.

15.1 Scalability: handling 100x growth in documents and traffic

At 100x scale, the major stress points are API concurrency, Kafka throughput, Elasticsearch index growth, Redis memory pressure, and PostgreSQL write/read contention. The architecture is already decomposed in a way that supports horizontal scaling, so the production plan would focus on scaling each tier independently rather than vertically scaling a single node.

API and application layer

The Spring Boot API should remain stateless so that it can scale horizontally behind a load balancer or Kubernetes service. Autoscaling would be driven by CPU, memory, request latency, and queueing behavior. Read-heavy search traffic and write-heavy document ingestion traffic would ideally be handled by separate autoscaling profiles, because they stress different downstream systems.

Kafka and indexing pipeline

The current prototype uses a single topic partition for deterministic behavior, but at scale I would move to a partition count sized for projected ingestion throughput. A rough production pattern would be:

- partition by document ID hash or tenant-aware routing
- multiple consumer instances in the same group
- lag-based autoscaling for consumers
- dead-letter topic for poison messages
- idempotent indexing so replay is safe

This ensures that indexing throughput scales with traffic while keeping failures isolated.

Elasticsearch

Elasticsearch is the most important scale tier for query latency. To handle 10M+ documents and high query concurrency, I would:

- use multiple primary shards sized according to index size and expected write rate
- configure replicas for read scaling and high availability
- separate hot and warm tiers if document access is skewed
- tune refresh interval depending on indexing pressure
- use index lifecycle management for aging data if retention becomes large

I would also evaluate index-per-environment and possibly index-per-large-tenant patterns if a few tenants dominate footprint or query load. For smaller tenants, shared multi-tenant indices with tenant filtering are cost-efficient; for very large tenants, isolated indices can improve operational control.

PostgreSQL

PostgreSQL remains the system of record, so it must scale for durable writes and point reads by ID. At 100x scale, I would:

- use connection pooling aggressively
- add read replicas if get-by-ID traffic becomes large
- partition data by tenant or time if table size becomes operationally heavy
- keep document metadata and document body storage under review if body size grows materially
- consider a separate archival strategy for cold data

The key principle is that PostgreSQL should not become the search engine; it should remain optimized for correctness and transactional durability.

Redis

Redis use would expand from a single node to a replicated or clustered setup depending on query volume. Search cache TTLs would remain short to avoid large stale working sets. Cache admission should remain selective; hot repeated queries provide the best ROI, while low-frequency searches do not justify large memory usage.

Traffic management

To protect the system under rapid growth, I would combine:

- per-tenant rate limits
- per-user/API token quotas
- request size limits
- backpressure on write ingestion
- queue lag alerts and autoscaling

This prevents noisy-neighbor behavior and protects cluster stability.

15.2 Resilience: circuit breakers, retries, and failover mechanisms

A production-ready system must assume partial failure is normal: Kafka brokers may lag, Elasticsearch nodes may reject requests, Redis may fail over, and downstream dependencies may become slow before they become unavailable.

Circuit breakers

The API layer should wrap Elasticsearch and Redis calls with circuit breakers and bounded timeouts. If Elasticsearch becomes slow, the system should fail fast rather than hanging request threads and causing cascading failure. For example:

- short timeout on Elasticsearch query calls
- short timeout on Redis lookups
- fallback to uncached search if Redis is unavailable
- degrade gracefully when highlight/fuzzy features become too expensive

Retry strategies

Retries should be applied selectively:

- synchronous API requests: very limited retries, only for clearly transient failures
- Kafka consumers: retry with exponential backoff for transient indexing failures
- dead-letter queue for non-recoverable events
- idempotent delete/upsert handling so replay is safe

One important production rule is to avoid blind retries that amplify load during an outage. Retries should be jittered, bounded, and observable.

Failover mechanisms

- PostgreSQL: managed HA with primary/standby failover, PITR backups, and replica promotion
- Redis: Sentinel or managed Redis failover
- Kafka: multi-broker cluster with replication factor > 1
- Elasticsearch: multi-node cluster with replicas and shard allocation awareness
- API: multi-instance stateless deployment across availability zones

Data consistency under failure

The write path should move toward an **outbox pattern** or CDC-based event publication. The prototype uses flush-before-publish to avoid a transaction visibility race, but in production I would want atomic persistence of both:

1. database write
2. “index this document” intent

An outbox table consumed asynchronously is the cleanest production answer because it avoids dual-write inconsistency.

15.3 Security: authn/authz, encryption, and API security

The prototype demonstrates tenant isolation, but production security requires both identity and policy enforcement.

Authentication

I would place the API behind an identity-aware gateway or directly integrate JWT/OAuth2. Each request would carry:

- authenticated principal
- tenant membership / scopes
- possibly service-to-service identity for internal calls

Authorization

Tenant ID must not be accepted as a free-form trust signal. In production, the server should derive or validate tenant context from authenticated claims. Authorization policy should ensure:

- user belongs to tenant
- caller has permission for read/write/delete
- admin and service roles are separated
- audit trail exists for document mutations

Encryption

- TLS in transit everywhere: client-to-API and service-to-service where applicable
- encryption at rest for PostgreSQL, Redis, Kafka disks, and Elasticsearch data volumes
- secret management through a vault or cloud secret manager
- rotated credentials and limited-scope service accounts

API security

- schema validation and size limits on document payloads
- strict content-type validation
- abuse protection via rate limits and quotas
- WAF/API gateway protections
- request logging with PII-aware redaction

Search safety

Because search results can expose sensitive content, I would also add:

- field-level redaction if needed
- audit logs for searches in regulated environments
- optional result filtering by document classification/tag

15.4 Observability: metrics, logging, and distributed tracing

A distributed search system is only production-ready if it is easy to observe and easy to debug under partial failure.

Metrics

I would instrument the following core metrics:

API metrics

- request rate
- latency p50/p95/p99 by endpoint
- error rate by endpoint and tenant class
- payload sizes
- throttle rates

Kafka metrics

- publish success/failure
- topic lag by partition
- consumer throughput
- retry count
- dead-letter volume

Elasticsearch metrics

- query latency
- indexing latency
- rejection rate
- shard health
- cluster state / replica health
- cache hit ratios where relevant

Redis metrics

- hit/miss rate
- memory utilization

- evictions
- command latency
- connection errors

PostgreSQL metrics

- write latency
- read latency
- lock contention
- connection pool saturation
- slow query counts
- replication lag if replicas exist

Logging

Logs should be structured JSON with correlation fields such as:

- request ID / trace ID
- tenant ID
- document ID
- event ID
- endpoint
- status code
- dependency status and latency

This makes it much easier to trace a document across:

- API write
- Kafka publish
- consumer processing
- Elasticsearch indexing
- search retrieval

Distributed tracing

Distributed tracing should be added across:

- inbound HTTP request
- Kafka publish/consume
- Elasticsearch query
- Redis lookup
- PostgreSQL persistence

This is especially valuable for debugging indexing lag, search slowness, and intermittent dependency issues.

Alerting

I would alert on:

- sustained p95 latency breaches
- rising 5xx rates
- Kafka lag growth
- Elasticsearch cluster yellow/red state
- Redis memory pressure
- PostgreSQL pool saturation
- high throttle rates
- indexing success rate drops

15.5 Performance: DB optimization, index management, and query tuning

PostgreSQL optimization

The primary DB workload should remain simple and predictable:

- primary key lookup by document ID
- write/update/delete operations
- tenant ownership checks

Production improvements would include:

- appropriate indexes on (id) and (tenant_id, id) if needed
- connection pool tuning
- careful transaction boundaries
- batched writes for ingestion pipelines if bulk APIs are added

Elasticsearch index management

Performance at scale depends heavily on sensible index design:

- right shard count, not over-sharding
- controlled replica count
- tuned refresh interval for indexing-heavy bursts
- bulk indexing where applicable
- separate analyzers if language/tokenization needs evolve

Query optimization

The current query is intentionally simple: multi-match on title and content with title boost and tenant filter. Production tuning would likely add:

- better analyzers
- query rewriting for precision/recall balance
- field weighting by document type
- stop-word and synonym handling
- optional rescore phase for top-N results
- selective use of fuzziness to avoid cost blowups on every query

Cache optimization

Redis should be used where it helps most:

- repeated tenant-scoped searches
- short TTL
- query normalization to improve hit rate
- no caching of empty results in eventually consistent flows

Benchmarking

Before declaring production readiness, I would run:

- load tests with realistic query mix
- cache-hit vs cache-miss comparisons
- write burst tests
- failure injection during search and indexing
- long-duration soak tests

15.6 Operations: deployment, zero-downtime updates, and backup/recovery

Deployment strategy

For production I would use containerized deployment on Kubernetes or an equivalent orchestrator. The API tier is a straightforward rolling deployment candidate because it is stateless.

Zero-downtime releases

For safer releases, I would use:

- rolling deployments for backward-compatible changes
- blue-green or canary deployments for higher-risk releases
- health-based promotion gates
- smoke tests before traffic cutover

Database and schema changes

Schema migrations must be backward compatible. The safest pattern is:

1. add new columns/indexes in a backward-compatible way
2. deploy code that can handle both old and new states
3. backfill if needed
4. remove legacy behavior later

Backup and recovery

- PostgreSQL: scheduled backups + point-in-time recovery

- Elasticsearch: snapshot repository for restore
- Kafka: replicated brokers plus retention policy tuned for replay needs
- Redis: depending on cache-only vs durable limiter needs, persistence policy aligned to role

Disaster recovery

At higher maturity, I would add:

- documented RTO/RPO targets
- restore drills
- region-level failover design for critical deployments
- infra-as-code for reproducible rebuilds

15.7 SLA considerations: achieving 99.95% availability

99.95% monthly availability allows roughly **~22 minutes of downtime per month**, so this target requires disciplined operational design.

Availability strategy by tier

- API: multiple stateless instances across zones
- PostgreSQL: HA primary + standby, automated failover
- Redis: replicated/failover setup
- Kafka: multi-broker replicated cluster
- Elasticsearch: replicas across nodes/zones

Reducing downtime risk

- zero-downtime deployments
- short, bounded timeouts on dependencies
- circuit breakers
- autoscaling before saturation
- aggressive alerting on early warning signals
- regular disaster-recovery drills

Service degradation model

To hit 99.95%, the system should degrade gracefully:

- if Redis fails, search still works without cache
- if Elasticsearch is degraded, CRUD path still works because Postgres remains canonical
- if Kafka lags, documents may be delayed in search but writes still succeed
- if one consumer fails, lag alerts fire and other instances take over

SLO thinking

I would define at least these SLOs:

- Search latency p95 < 500 ms
- Search success rate > 99.9%
- Document create durability > 99.99%
- Indexing freshness target (e.g. 95% of documents searchable within X seconds)
- Kafka lag below an operational threshold

This is important because search correctness is not just “service up/down”; freshness is part of the user experience.

Enterprise Experience Showcase (Deep Distributed Systems)

1. Similar Distributed System

I architected a **multi-tenant, event-driven indexing and query platform** for blockchain data across Ethereum and Tendermint-based chains.

From a distributed systems perspective:

- Data ingestion was **append-only event streams** (block events, transactions)
- Used Kafka as a **durable log + replayable source of truth**
- Built **separate read models** optimized for:
  - real-time queries (low latency)
  - analytics (batch pipelines)

Key challenges:

- **High write throughput bursts** (block production spikes)
- **Out-of-order events** due to network propagation delays
- **Eventual consistency vs query freshness trade-offs**

Solutions:

- Introduced **idempotent consumers** (dedupe by tx hash)
- Designed **deterministic replay pipelines** for recovery
- Partitioned Kafka by chain + contract → ensured ordering where needed

Impact:

- Scaled to **hundreds of millions of events/day**
- Achieved **sub-200ms query latency**
- Enabled near real-time analytics dashboards

---

2. Performance Optimization

I optimized a large-scale content analytics system.

Problem:

- Heavy read amplification:
  - Same queries hitting DB repeatedly
- Query latency: **2–3 seconds**

Distributed systems issues:

- DB became a **central bottleneck**
- Lack of **read isolation layer**
- Poor cache locality

Solution:

- Introduced:
  - **Redis-based read-through caching**
  - **Kafka-driven async aggregation pipeline**
- Shifted from:
  - synchronous query computation
    → **precomputed materialized views**

Key insight:

- Move from **compute-on-read → compute-on-write**

Impact:

- Latency reduced **2.5s → ~200ms**
- DB load reduced ~70%
- System became horizontally scalable

---

3. Critical Production Incident

We faced a **consensus-level failure (chain halt)**.

Root cause:

- Bug in **BeginBlock reward distribution logic**
- Caused:
    - inconsistent state transitions across validators
    - consensus failure

Distributed systems challenge:

- System required **strong consistency across nodes**
- No partial recovery possible (all validators must agree)

Resolution:

- Used **deterministic replay of blocks** to isolate divergence
- Introduced:
    - feature flags to disable faulty logic
    - patch deployment across validator set

Key learning:

- Need for:
    - **deterministic execution guarantees**
    - observability inside consensus-critical paths

Post-fix improvements:

- Added:
    - replay tooling
    - invariants validation
    - enhanced telemetry on state transitions

---

4. Architectural Trade-off Decision

While designing liquid staking systems at
Stride and Quicksilver:

Core decision:
Strong consistency vs Eventual consistency

Context:

- Cross-chain operations (IBC)
- Network latency + partial failures expected

Option 1:

- Strong consistency:
    - tightly coupled chains
    - blocking operations
    - poor availability

Option 2 (chosen):

- Event-driven async model:
    - eventual consistency
    - reconciliation-based correctness

Implementation:

- Used:
    - **event sourcing pattern**
    - **compensating transactions**
- Built reconciliation loops:
    - periodically verify state across chains

Impact:

- System remained:
    - **highly available**
    - resilient to partial failures

- Enabled:
    - scaling across multiple chains independently